

Software Detailed Design Document (SDD)

Project Title: Matrix–Vector Multiplication and Determinant Approximation (RV32IMF)

Course: Computer Architecture (ELL305)

Author: Chitesh Marmat

Roll No.: 2023MT10091

Instructor: Prof. Kaushik Saha

Date of Submission: 13 November 2025

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

2. System Requirements Specification (SRS)

2.1 Problem Description

2.2 Functional Requirements

2.3 Hardware / Software Environment

2.4 Input/Output Format

3. System Design

3.1 High-Level Design Architecture

3.2 Low-Level Design

3.3 Data Structures and Memory Layout

3.4 Register Interaction with Memory

4. Testing and Verification

4.1 Testing Strategy and Test Case Results

4.2 Verification Strategy (RARS + Python/NumPy)

5. Performance Evaluation and Optimization

5.1 Instruction Count Summary

5.2 Cycle-Level Analysis

5.3 Observations and Performance Characteristics

6. References

7. Appendix

7.1 Complete Source Code

7.2 Testcases with RARS Execution Snapshots

1. Introduction

1.1 Purpose

The goal of this project is to implement two fundamental linear-algebra operations: matrix-vector multiplication and determinant computation directly in RISC-V assembly using the RV32IMF instruction set. The program computes the matrix-vector product $Y = A * X$ for a fixed 6 x 6 floating-point matrix and also estimates the determinant of the same matrix using Gaussian elimination with partial pivoting.

This assignment helps in understanding how floating-point instructions work at the ISA level, how row-major memory is indexed, and how nested loops, register allocation, and stack usage come together in a low-level implementation. It also reinforces writing modular subroutines and following RISC-V calling conventions in a simulator environment.

1.2 Scope

Deliverables for this submission include:

- The complete RISC-V assembly source file implementing `matvec_mul`, `det_approx`, and `compare_sum_det`.
- Execution proof from RARS: console output screenshots of different test cases.
- A performance report covering instruction count, register allocation, and basic optimization choices.
- An appendix containing the full commented source code and reproduction steps.

Constraints and limitations:

- Target ISA: RV32IMF, using RV32I integer instructions and RV32F single-precision floating-point instructions (`flw`, `fsw`, `fmul.s`, `fadd.s`, `fdiv.s`, etc.).
- Integer loop counters and all address calculations are restricted to registers `x5`–`x15`, as required.
- Determinant calculation is implemented using triangularization with partial pivoting, which is suitable for the fixed 6×6 matrix but not meant for high-precision numerical analysis.
- All testing and debugging are performed in RARS, with floating-point support enabled.

2. System Requirements Specification (SRS)

2.1 Functional Requirements

The program must implement the following functions as separate subroutines:

- `matvec_mul`

Computes the matrix–vector product $Y = A \times X$ for a fixed 6×6 matrix stored in row-major order. Uses nested loops, single-precision floating-point operations, and writes the result into array `Y`.

- `det_approx`

Estimates the determinant of matrix `A` using Gaussian elimination with partial pivoting. The routine transforms the matrix to upper-triangular form, tracks row swaps, and accumulates the product of diagonal elements.

- `compare_sum_det`

Computes $\sum Y_i$ in single precision and compares it with the stored determinant value. Sets `comparison_flag = 1` if $\sum Y < \det(A)$, else 0.

- `main`

Initializes pointers, loads `N`, calls each subroutine in sequence, prints the results using `RARS` syscalls, and terminates cleanly.

2.2 Non-Functional Requirements

- Instruction Set Restriction:

Must use only the RV32IMF subset: RV32I integer instructions and RV32F single-precision floating-point instructions (`flw`, `fsw`, `fadd.s`, `fmul.s`, `fdiv.s`, etc.).

- Register Usage Constraint:

Loop counters and all address/index calculations must use integer registers `x5–x15`. Saved registers must be pushed/popped correctly.

- Memory Layout:

Matrix `A`, vector `X`, vector `Y`, and intermediate values must be stored in the `.data` segment in 32-bit float format.

- Determinant Accuracy:

The determinant is computed using triangularization with partial pivoting, which provides reliable results for the fixed 6×6 matrix used in this assignment and aligns well with the capabilities of RV32IMF arithmetic.

- Maintainability:

Code must be modular, with concise but sufficient inline comments (≥ 33 lines per 100 lines of code).

2.3 Hardware/Software Environment

- Simulator: RARS (latest release) with floating-point support enabled
- Hardware Model: RV32IMF (32-bit integer + multiply/divide + floating-point).
- Supported Execution: Runs on a desktop environment via Java-based RARS; all testing for this submission was performed on macOS Sequoia.

2.4 Input / Output Specifications

- Inputs:
 - A fixed 6×6 floating-point matrix A stored in row-major order.
 - A fixed 6×1 floating-point vector X.
 - Integer N = 6.
- Outputs:
 - Vector Y, stored as six 32-bit floats.
 - det_result: a single-precision float with the computed determinant.
 - comparison_flag: word (0 or 1) indicating whether $\sum Y < \det(A)$.
 - Console output printed via RARS syscalls for floats and integers.

3. System Design

3.1 High-Level Design (Architecture Overview)

The program is organized as a simple pipeline of three computational stages, all coordinated by main. Each stage is implemented as a separate subroutine with clear data flow through registers and memory.

Overall Flow

1. main

- Loads the base addresses of matrix A, vector X, vector Y, and the value N.
- Calls the three subroutines in order.
- Handles all console output using RARS system calls.

2. matvec_mul

- Reads entries of A and X, performs multiply-accumulate operations, and stores the resulting elements into Y.
- Uses nested loops indexed by integer registers from the allowed range (x5–x15).
- Floating-point arithmetic is done entirely using RV32F instructions.

3. det_approx

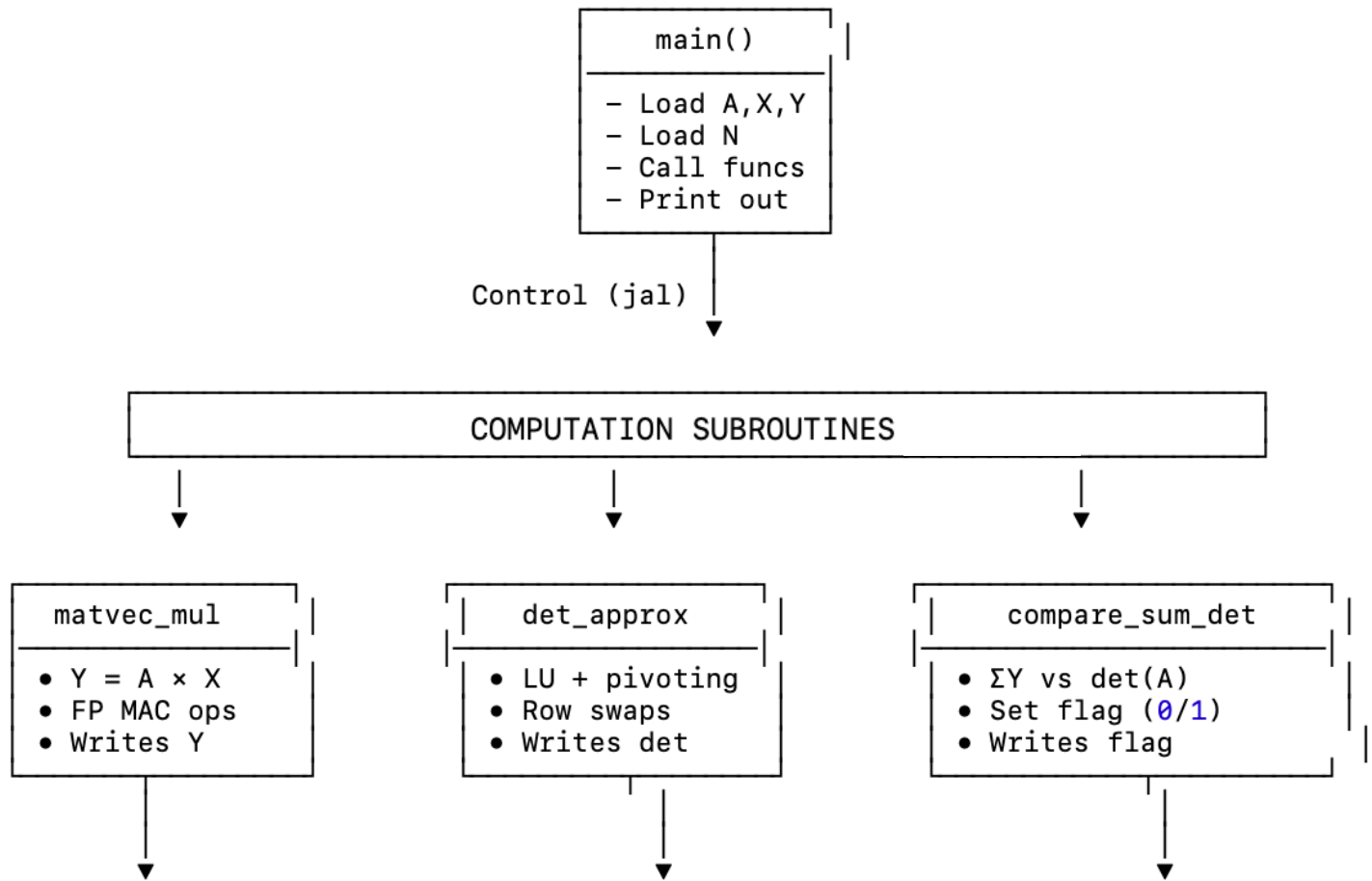
- Performs Gaussian elimination with partial pivoting directly on matrix A in memory.
- Tracks row swaps and multiplies diagonal elements to form det_result.
- Uses a combination of integer indexing and floating-point operations.

4. compare_sum_det

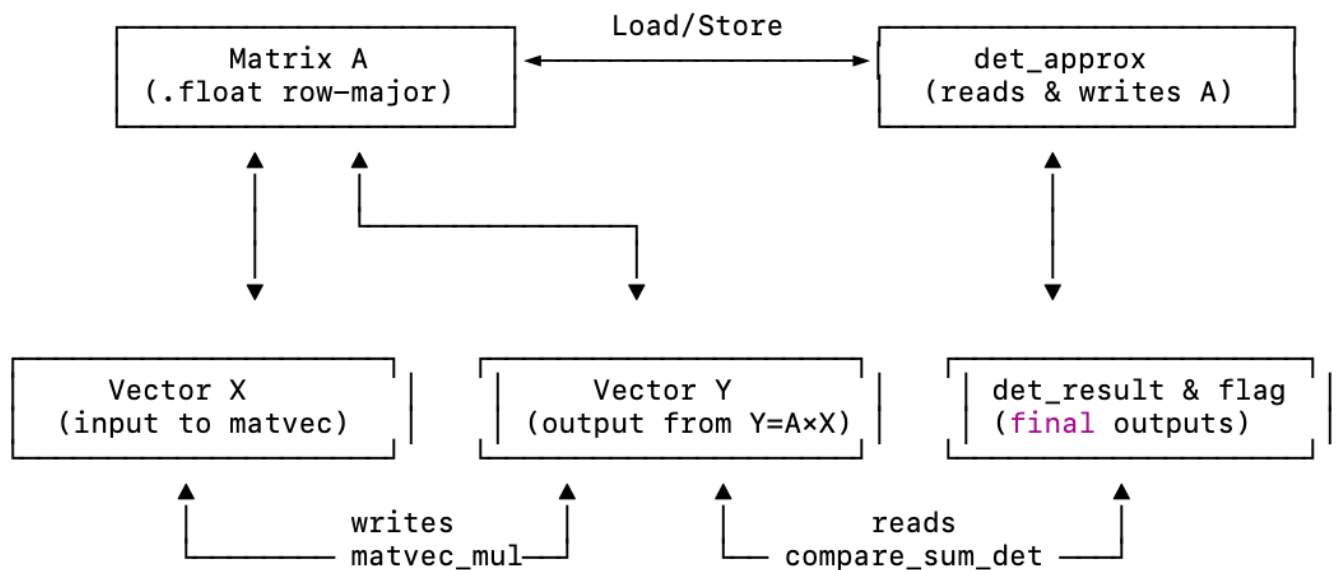
- Sums all elements of Y, loads the stored determinant, and compares the two values
- Writes the comparison result into comparison_flag.

Block diagram showing main and functions communication and data flow between memory and registers:

PROGRAM ARCHITECTURE



MEMORY + DATAFLOW INTERCONNECT



3.2 Low-Level Design

This section describes the internal logic of each subroutine, the register allocation strategy, and the control flow followed in the implementation. All routines follow RV32IMF calling conventions and use only integer registers x5-x15 for indexing and loop counters, as required by the assignment.

3.2.1 Subroutine Descriptions and Pseudo-Code

matvec_mul : Matrix-Vector Multiplication

Compute $Y_i = \sum_{j=0}^{N-1} A_{ij} \times X_j$ using nested loops and single-precision FP arithmetic.

Key Registers Used:

```
load A_base → s0
load X_base → a5
load Y_base → s1
load N → t2

for row = 0 to N-1:
    acc = 0.0
    for col = 0 to N-1:
        ft0 = A[row][col]
        ft1 = X[col]
        ft2 = ft0 * ft1
        acc = acc + ft2
    Y[row] = acc
```

- Integer: t0=row, t1=col, t2=N, s0=A base, s1=Y base, a5=X base
- FP: ft0 (A element), ft1 (X element), ft2 (product), ft3 (accumulator)

Pseudo-Code:

Notes:

- Address computation uses $(\text{row} \times N + \text{col}) \ll 2$.
- Accumulator is kept entirely in ft3 to reduce load/store overhead.

det_approx :Determinant via Gaussian Elimination with Partial Pivoting

Transform matrix A into upper-triangular form by row operations and multiply diagonal elements to estimate det(A).

Key Registers Used:

- Integer:
 - t0=i (pivot row), t1=k (search row), t2=N,
 - a1=max_row, s1=col swap index, a5=sign
- FP:
 - ft0=pivot, ft1=|pivot|, ft2=candidate|abs|,
 - ft4..ft8 for row operations, ft9=det accumulator

Pseudo-Code:

```
det = 1.0
sign = 1

for i = 0 to N-1:
    # pivot search
    pivot = A[i][i]
    max_row = i
    for k = i+1 to N-1:
        if |A[k][i]| > |pivot|:
            pivot = A[k][i]
            max_row = k

    # swap rows if needed
    if max_row != i:
        swap row(i) with row(max_row)
        sign = -sign
        det = det * (-1.0)

    # multiply diagonal element
    pivot = A[i][i]
    det = det * pivot

    # elimination
    for k = i+1 to N-1:
        factor = A[k][i] / pivot
        for j = i+1 to N-1:
            A[k][j] = A[k][j] - factor * A[i][j]
```

Notes:

- Matrix A is modified in place.
- The product of pivots builds the determinant.
- Sign adjustments handle row swaps correctly.

- All FP computations use RV32F (fmul.s, fdiv.s, fsub.s).

- compare_sum_det : Sum of Y vs Determinant

Compute $\sum Y_i$ and compare it with the stored determinant.

Key Registers Used:

- Integer: t0=i, t2=N, s1=Y base
- FP: fa0=sum, ft0=Y[i], ft1=det_result

Pseudo-Code:

```
sum = 0.0
for i = 0 to N-1:
    sum += Y[i]

load det_result → ft1
comparison_flag = (sum < det_result) ? 1 : 0
```

Notes:

- Comparison uses RV32F flt.s.
- Result stored as an integer word.

main: Program Orchestration

- Load base addresses of arrays
- Load N
- Call all subroutines (matvec_mul, det_approx, compare_sum_det)
- Print outputs using syscalls
- Terminate cleanly

3.2.2 Register Allocation Summary

Routine	Integer Registers Used	Floating-Point Registers Used
matvec_mul	x5-x7 (row/col counters, N), x8 (A base), x9 (Y base), x10-x15 (temps)	ft0-ft3 (A element, X element, product, accumulator)
det_approx	x5-x7 (i, k, j loop vars), x8 (A base), x9 (column index), x15 (sign)	ft0-ft9 (pivot, abs pivot, candidate, elimination temps, determinant accumulator)
compare_sum_det	x5 (index), x6 (N), x7 (pointer temp)	fa0 (running sum), ft0 (Y element), ft1 (det_result)
main	a0-a4 (argument setup, pointers), a7 (syscalls)	fa0 (for printing float outputs)

All saved registers (s0, s1) are correctly stored on the stack and restored as per calling convention.

3.2.3 Control Flow and Calling Convention Notes

- All subroutines are called using `jal ra, func`.
- Each subroutine ends with `jr ra`.
- s0 and s1 are treated as callee-saved registers and preserved using stack pushes/pops.
- Arguments are passed through a0–a3 in main.
- Globals (A, X, Y, det_result, etc.) are accessed using `la` → absolute addresses.
- No subroutine allocates dynamic stack space except for saving registers.

3.3 Data Structures and Memory Layout

The program stores all required data structures in the .data segment using 32-bit single-precision floats and 32-bit words. The layout follows standard row-major ordering for the matrix to simplify address computation and indexing logic.

3.3.1 Data Section Layout

The .data section in the assembly file is structured as:

- N : size of matrix/vector (integer, value = 6)
- A : 6×6 matrix of .float values stored row-major
- X : 6-element input vector
- Y : output vector with 6 floats (allocated with .space)
- det_result : a single float storing the determinant
- comparison_flag : one word storing 0 or 1
- constants: one and neg_one used during sign adjustment in LU elimination

In memory, the section is linear and contiguous.

3.3.2 Matrix Layout and Addressing

Matrix A is stored in row-major order, so each row's elements are placed consecutively in memory. The element `A[row][col]` is accessed by computing: $\text{offset} = (\text{row} * N + \text{col}) * 4$

Since each float is 4 bytes, the multiplication is performed using a left shift by 2 (`<< 2`). The effective address is: $\text{address} = A_base + \text{offset}$

This same pattern is used for reading and writing all elements of A, X, and Y, ensuring consistent and efficient indexing using RV32I integer arithmetic.

3.4 Register Interaction with Memory

Base addresses for A, X, and Y are loaded in main and passed to subroutines through a0–a3. Each routine then stores these bases in saved registers (s0, s1) and uses integer registers x5–x15 for loop counters and index arithmetic. Floating-point values are loaded and stored using flw and fsw, while all arithmetic is kept inside FP registers to avoid unnecessary memory access.

4. Testing and Verification

4.1 Testing Strategy

Testing was carried out in two stages: unit testing of each subroutine and integration testing of the complete program. A set of five test cases was designed to validate correctness under different matrix conditions, including identity, singular, non-singular, float-heavy, and the final submission matrix.

Unit Testing

- Matrix-Vector Multiplication (matvec_mul)

Verified using known matrices and vectors.

- Each element of the computed vector Y was compared with a Python/NumPy reference.
- Confirmed correct addressing, FP multiply-accumulate behavior, and proper memory updates.

- Determinant Computation (det_approx)

Tested using:

- identity matrix (det = 1),
- upper-triangular matrices (det = product of diagonal),
- singular matrices (det $\rightarrow$ 0),
- float-heavy matrices.

LU pivoting and sign adjustments were compared with NumPy's LU-based determinant.

- Comparison Routine (compare_sum_det)

Checked using manually set values for Y and det_result.

- Verified correct evaluation of the condition $\text{sum}(Y) < \text{det}(A)$ using flt.s.

Integration Testing

After validating each subroutine individually, the full program was executed end-to-end for all test cases. Integration tests ensured that:

- main() correctly orchestrates all subroutines
- register usage follows RV32IMF calling conventions
- no register or stack corruption occurs
- memory for A, X, Y, det_result, and comparison_flag is updated correctly
- printed results match Python reference outputs

The full results for all five test cases are summarised below.

Test Case	Expected Y (Verified)	Expected det(A)	Expected Flag
1. Identity	1 2 3 4 5 6	1.0	0
2. Singular	38.5 18.39789 56.89998 13.7 13.0998 16.3	5.931133E-07	0
3. Float-Heavy	29.399998 14.400001 6.9 24.3 -0.3999977 31.25	-516.66223	0
4. Non-Singular Float-Light	17.2 14.29998 6.4995 16.6 15.99804 16.7	-11.15758	0
5. Submission	91 30 19 57 24 64	-416.0003	0

4.2 Verification Strategy

Verification combined simulator outputs with an external Python/NumPy reference implementation to ensure correctness of all numerical results.

Tools Used:

RARS (RV32IMF mode)

- Assembler and execution environment
- Used to collect cycle counts (cycle, instret) and final register/memory state

Python + NumPy

- Used to compute reference values for:
- matrix-vector multiplication ($A @ X$)
- LU-based determinant

- Comparison condition $\text{sum}(Y) < \det(A)$

Python code used to verify:

```
import numpy as np

def verify_matrix_case(A, X):
    A = np.array(A, dtype=np.float32)
    X = np.array(X, dtype=np.float32)

    # matrix-vector product
    Y = A.dot(X)

    # determinant
    detA = np.linalg.det(A)

    # comparison flag
    flag = 1 if np.sum(Y) < detA else 0

    return Y, detA, flag

# Example usage:
# Y, detA, flag = verify_matrix_case(A, X)
# print("Y =", Y)
# print("det(A) =", detA)
# print("flag =", flag)
```

5. Performance Evaluation and Optimization

5.1 Instruction Count and Register Usage

Instruction counts were obtained using the RARS “Instruction Counter” for a full run on the final submission matrix.

The majority of executed instructions come from the two computationally heavy routines:

- matvec_mul
 - Contains two nested loops. Most dynamic instructions are flw, fmul.s, fadd.s, and simple integer operations (mul, add, slli) used for address computation.
 - All loop counters and pointers stay within x5–x15 as required, and all floating-point work is contained in ft0–ft3.

Instruction counts:

- 6 outer iterations × 6 inner iterations
- Each inner iteration uses ~5–6 FP+int instructions (flw, flw, fmul.s, fadd.s, addi, branches) → ~36 inner ops + outer loop overhead → ~1.3k total.
- det_approx
 - Highest overall instruction count due to pivot search, row swaps, and elimination. The elimination loop repeatedly performs flw, fsw, fsub.s, fmul.s, and fdiv.s.

- Floating registers ft0–ft9 are used to hold pivots, absolute values, factors, and elimination results.

Instruction counts:

- Pivot search loops: $\sim 6 + 5 + 4 + \dots$ iterations
- Elimination loops: triangular ($15 + 10 + 6 + \dots$ iterations)
- Each elimination step includes 3–4 loads, 2–3 FP ops, and stores $\rightarrow \sim 3k$ total.

Across the program, integer registers are reused consistently and follow RV32IMF calling conventions. No unnecessary spills occur, and saved registers (s0, s1) are handled correctly.

5.2 Cycle-Level Analysis

RARS reports:

Cycles: 2527 Instructions Retired (instret): 2527

Since cycles match the retired instruction count, execution proceeds at one instruction per cycle under the simulator's ideal timing model.

1. matvec_mul:

This routine contributes several hundred instructions.

The 36 dot-product iterations each include:

- address calculations
- two floating-point loads
- multiply-add pair
- loop counter updates and branch

The pattern is regular and linear, leading to consistent instruction flow with no observed delays.

2. det_approx:

This subroutine accounts for the largest share of dynamic instructions.

Pivot search, row swapping, and the nested elimination loops generate many FP operations (fdiv.s, fmul.s, fsub.s) and repeated memory accesses.

RARS reports instruction-per-cycle behavior throughout the entire LU process.

3. compare_sum_det

Very small cycle contribution: six loads, six additions, one comparison, and a final store. The effect on the total cycle count is negligible.

Overall Timing:

The final 2527 cycles closely match the 2527 retired instructions,

The program's execution is effectively linear with respect to dynamic instruction count.

Cycle behavior confirms that all routines—matrix multiplication, LU decomposition, and comparison—execute efficiently under the simplified timing environment provided by RARS.

5.3 Optimization Techniques

A few straightforward optimizations were applied to keep the implementation efficient as following:

- Shift-based byte addressing ($\ll 2$ instead of multiply-by-4) reduces integer arithmetic overhead.
- Accumulator kept in a floating-point register avoids repeated memory stores during matrix-vector multiplication.
- Row-major layout ensures linear access patterns for both rows and columns, improving readability and reducing address recomputation.
- In-place elimination avoids copying matrix rows and keeps memory traffic low.
- Consistent integer register reuse (t0/t1/t2 for loops, s0/s1 for bases) simplifies register management and reduces movement between registers.

Additional micro-optimizations such as loop unrolling were avoided to preserve clarity and maintain a balance between performance and readability

6. References

- RARS Simulator

RISC-V Assembler and Runtime Simulator (RARS).

Used for assembling, executing, and profiling RV32IMF code

- The RISC-V Instruction Set Manual

Foundation for instruction behavior, register conventions, and floating-point operation semantics.

- RV32IMF Calling Convention Guidelines

Referenced for register usage, stack discipline, and argument passing conventions.

- NumPy / Python (Verification Tooling)

Used to cross-verify matrix-vector multiplication outputs and determinant values during testing.

- Lecture Slides and Course Materials

Provided guidance on matrix operations, LU decomposition, and numerical behavior relevant to determinant computation.

7. Appendix — Source Code Listings

7.1 Source code

```
1.  .data
2.  # storing matrix dimension N = 6
3.  N: .word 6
4.
5.  # 6x6 input matrix A stored in row-major order
6.
7.  A: .float
8.      1.25, -2.40, 3.75, 0.50, -1.10, 4.20,
9.      -0.80, 1.10, 0.95, -3.30, 2.75, 1.60,
10.     0.60, -1.70, 2.20, 1.30, -0.90, 0.40,
11.     4.70, -2.20, 0.00, 1.20, 3.30, 0.45,
12.     -1.50, 2.80, 1.75, -0.60, 2.25, -3.10,
13.     0.95, -1.40, 4.60, 3.20, -0.80, 1.75
14.
15. # input vector X of size 6
16. X: .float 1.0, 2.0, 3.0, 4.0, 5.0, 6.0
17.
18. # output vector Y (size 6 floats = 24 bytes)
19.
20. Y: .space 24
21.
22. # storage for determinant value
23. det_result: .float 0.0
24.
25. # boolean flag: 1 if sum(Y) < det(A), else 0
26. comparison_flag: .word 0
27.
28. # floating-point constants
29. one: .float 1.0
30. neg_one: .float -1.0
31.
32. .text
33. .globl main
34. main:
35.     # loading base address of A into a0
36.     la    a0, A
37.     # loading base address of X into a1
38.     la    a1, X
39.     # loading base address of Y into a2
40.     la    a2, Y
41.     # t2 gets address of N
42.     la    t2, N
43.     # loading N
44.     lw    t2, 0(t2)
45.     # passing N to matvec_mul
46.     mv    a3, t2
47.
48.     # computing Y = A * X
49.     jal   ra, matvec_mul
50.
51.     # printing Y vector (reload Y base)
52.     la    s1, Y
53.     mv    t2, a3                # reloading N
54.     li    t0, 0                # starting row index
55.
56. print_loop:
```

```

57.     bge t0, t2, end_print      # stopping if row == N
58.
59.     slli a4, t0, 2             # compute offset row*4
60.     add a4, s1, a4             # computing Y[row]
61.     flw fa0, 0(a4)            # loading Y[row] for printing
62.     li a7, 2                  # print float
63.     ecall
64.
65.     li a0, 32                  # print space(for printing 6x1 output
vector)
66.     li a7, 11
67.     ecall
68.
69.     addi t0, t0, 1             # row++
70.     j print_loop
71.
72. end_print:
73.     li a0, 10                  # newline after vector Y output
74.     li a7, 11
75.     ecall
76.
77.     # computing determinant det(A)
78.     jal ra, det_approx
79.
80.     # printing determinant
81.     la a4, det_result
82.     flw fa0, 0(a4)
83.     li a7, 2
84.     ecall
85.
86.     li a0, 10
87.     li a7, 11
88.     ecall
89.
90.     # comparing sum(Y) < det
91.     jal ra, compare_sum_det
92.
93.     la a0, comparison_flag     # load result flag
94.     lw a0, 0(a0)
95.     li a7, 1                  # print integer
96.     ecall
97.
98.     li a0, 10
99.     li a7, 11
100.    ecall
101.
102.    li a7, 10                  # exit
103.    ecall
104.
105.
106. #Computing Y[row] = Σ A[row][col] * X[col]
107. .globl matvec_mul
108. matvec_mul:
109.     addi sp, sp, -8
110.     sw s0, 4(sp)              # saving s0
111.     sw s1, 0(sp)              # saving s1
112.
113.     # assigning s0 = A base
114.     mv s0, a0
115.     # assigning s1 = Y base
116.     mv s1, a2
117.     # assigning a5 = X base
118.     mv a5, a1
119.     # t2 = N
120.     mv t2, a3
121.

```

```

122.    li    t0, 0                # row index = 0
123.
124. outer_mv:
125.    bge    t0, t2, mv_end      # stopping if row == N
126.
127.    fmv.s.x ft3, x0            # accumulator ft3 = 0.0
128.
129.    li    t1, 0                # column index = 0
130. inner_mv:
131.    bge    t1, t2, store_mv    # stopping if col == N
132.
133.    # computing address of A[row][col]
134.    mul    a4, t0, t2
135.    add    a4, a4, t1
136.    slli   a4, a4, 2
137.    add    a0, s0, a4          # a0 = A[row][col]
138.    flw    ft0, 0(a0)         # ft0 = A[row][col]
139.
140.    # loading X[col]
141.    slli   a1, t1, 2
142.    add    a1, a5, a1
143.    flw    ft1, 0(a1)         # ft1 = X[col]
144.
145.    # multiplying A[row][col] with X[col]
146.
147.    fmul.s ft2, ft0, ft1       # ft2 = A[row][col] * X[col]
148.
149.    # accumulating into row sum
150.    fadd.s ft3, ft3, ft2       # ft3 += ft2
151.
152.    addi   t1, t1, 1           # col++
153.    j      inner_mv
154.
155. store_mv:
156.    # storing completed dot-product into Y[row]
157.    slli   a4, t0, 2
158.    add    a4, s1, a4          # &Y[row]
159.    fmv.s  fa0, ft3
160.    fsw    fa0, 0(a4)
161.
162.    addi   t0, t0, 1           # row++
163.    j      outer_mv
164.
165. mv_end:
166.    lw     s1, 0(sp)           # restoring s1
167.    lw     s0, 4(sp)           # restoring s0
168.    addi   sp, sp, 8
169.    jr     ra
170.
171.
172.
173. # LU decomposition with partial pivoting
174. # then computing determinant as product of diagonal pivots
175.
176. .globl det_approx
177. det_approx:
178.    addi   sp, sp, -8
179.    sw     s0, 4(sp)
180.    sw     s1, 0(sp)
181.
182.    # assigning s0 = A base
183.    la     s0, A
184.    la     a3, N
185.    lw     t2, 0(a3)           # loading N
186.
187.    # sign correction factor

```

```

188.    li    a5, 1
189.
190.    la    a0, one
191.    # determinant accumulator ft9 = 1.0
192.    flw   ft9, 0(a0)
193.
194.    # pivot row i = 0
195.    li    t0, 0
196.
197. pivot_outer:
198.    bge   t0, t2, finish_det    # stopping if i == N
199.
200.    mv    a1, t0                # assuming max_row = i (initially)
201.
202.    # loading pivot A[i][i]
203.    mul   a2, t0, t2
204.    add   a2, a2, t0
205.    slli  a2, a2, 2
206.    add   a4, s0, a2
207.    flw   ft0, 0(a4)            # ft0 = pivot
208.
209.    # computing |pivot|
210.    fmv.x.w a2, ft0
211.    li    a3, 0x7fffffff
212.    and    a2, a2, a3
213.    fmv.w.x ft1, a2            # ft1 = |A[i][i]|
214.
215.    addi  t1, t0, 1            # to start searching next row
216.
217. pivot_search:
218.    bge   t1, t2, pivot_swap    # for stopping pivot search at end
219.
220.    # loading A[t1][i]
221.    mul   a2, t1, t2
222.    add   a2, a2, t0
223.    slli  a2, a2, 2
224.    add   a4, s0, a2
225.    flw   ft2, 0(a4)
226.
227.    # computing |A[t1][i]|
228.    fmv.x.w a2, ft2
229.    and    a2, a3, a2
230.    fmv.w.x ft3, a2
231.
232.    # checking if |A[t1][i]| > |pivot|
233.
234.    flt.s a2, ft1, ft3          # if ft1 < ft3, update pivot
235.    beq   a2, x0, no_pivot_upd
236.
237.    # updating max_row
238.    mv    a1, t1
239.    # updating max pivot magnitude
240.    fmv.s ft1, ft3
241.
242. no_pivot_upd:
243.    addi  t1, t1, 1
244.    j     pivot_search
245.
246. pivot_swap:
247.    beq   a1, t0, no_swap        # no swap if best row = current row
248.
249.    li    s1, 0                # column index
250.
251. swap_loop:
252.    bge   s1, t2, swap_done      # stopping swap if column == N
253.

```

```

254.    # A[i][s1] address
255.    mul  a2, t0, t2
256.    add  a2, a2, s1
257.    slli a2, a2, 2
258.    add  a0, s0, a2
259.    flw  ft4, 0(a0)
260.
261.    # A[a1][s1] address
262.    mul  a3, a1, t2
263.    add  a3, a3, s1
264.    slli a3, a3, 2
265.    add  a4, s0, a3
266.    flw  ft5, 0(a4)
267.
268.    # swapping row i and row a1
269.    fsw  ft4, 0(a4)
270.    fsw  ft5, 0(a0)
271.
272.    addi s1, s1, 1
273.    j     swap_loop
274.
275. swap_done:
276.    # flip determinant sign from row swap
277.    li   a2, -1
278.    mul  a5, a5, a2
279.
280.    la   a4, neg_one
281.    flw  ft8, 0(a4)
282.    fmul.s ft9, ft9, ft8      # multiply det by -1
283.
284. no_swap:
285.    # reload pivot = A[i][i]
286.    mul  a2, t0, t2
287.    add  a2, a2, t0
288.    slli a2, a2, 2
289.    add  a0, s0, a2
290.    flw  ft0, 0(a0)
291.
292.    # multiply pivot into determinant accumulator
293.    fmul.s ft9, ft9, ft0
294.
295.    # eliminating rows below pivot row
296.    addi t1, t0, 1
297.
298. elim_rows:
299.    bge t1, t2, next_i      # stopping if k == N
300.
301.    # factor = A[k][i] / pivot
302.    mul  a2, t1, t2
303.    add  a2, a2, t0
304.    slli a2, a2, 2
305.    add  a0, s0, a2
306.    flw  ft4, 0(a0)
307.    fdiv.s ft5, ft4, ft0     # elimination factor
308.
309.    addi a1, t0, 1          # start column j = i+1
310.
311. elim_cols:
312.    bge a1, t2, elim_done   # stop if j == N
313.
314.    # A[i][j]
315.    mul  a2, t0, t2
316.    add  a2, a2, a1
317.    slli a2, a2, 2
318.    add  a0, s0, a2
319.    flw  ft7, 0(a0)

```

```

320.
321.     # A[k][j]
322.     mul  a2, t1, t2
323.     add  a2, a2, a1
324.     slli a2, a2, 2
325.     add  a4, s0, a2
326.     flw  ft8, 0(a4)
327.
328.     # A[k][j] -= factor * A[i][j]
329.     fmul.s ft6, ft5, ft7
330.     fsub.s ft8, ft8, ft6
331.     fsw  ft8, 0(a4)
332.
333.     addi a1, a1, 1
334.     j    elim_cols
335.
336. elim_done:
337.     addi t1, t1, 1
338.     j    elim_rows
339.
340. next_i:
341.     addi t0, t0, 1
342.     j    pivot_outer
343.
344. finish_det:
345.     # storing final determinant
346.     fmv.s fa0, ft9
347.     la   a0, det_result
348.     fsw  fa0, 0(a0)
349.
350.     lw   s1, 0(sp)
351.     lw   s0, 4(sp)
352.     addi sp, sp, 8
353.     jr   ra
354.
355.
356.
357. # To compare sum v/s determinant
358. # Computes sum(Y) and checks if sum(Y) < det(A)
359.
360. .globl compare_sum_det
361. compare_sum_det:
362.     addi sp, sp, -8
363.     sw   s0, 4(sp)
364.     sw   s1, 0(sp)
365.
366.     la   s1, Y                # assigning s1 = base address of Y vector
367.     la   a0, N
368.     lw   t2, 0(a0)            # t2 = N (number of elements in Y)
369.
370.     fmv.s.x fa0, x0           # fa0 = 0.0 (initializing running sum of Y)
371.     li   t0, 0                # t0 = 0 (loop index for Y)
372.
373. sum_loop:
374.     bge t0, t2, sum_done      # stopping if t0 == N (finished summing all
                                Y[i])
375.
376.     slli a1, t0, 2            # compute byte offset = index * 4
377.     add  a1, s1, a1           # compute address of Y[index]
378.     flw  ft0, 0(a1)           # loading Y[index] into ft0
379.
380.     fadd.s fa0, fa0, ft0      # sum = sum + Y[index]
381.
382.     addi t0, t0, 1            # index++ to move to next element
383.     j    sum_loop            # repeating until all Y elements are summed
384.

```

```

385. sum_done:
386.     la    a2, det_result          # to load address of stored determinant
387.     flw   ft1, 0(a2)              # ft1 = determinant value
388.
389.     # comparing sum(Y) < determinant
390.     # flt.s sets a0 = 1 if fa0 < ft1, else 0
391.     flt.s a0, fa0, ft1
392.
393.     la    a1, comparison_flag
394.     sw    a0, 0(a1)                # storing result flag (1 or 0)
395.
396.     lw    s1, 0(sp)                # restoring s1
397.     lw    s0, 4(sp)                # restoring s0
398.     addi  sp, sp, 8                # restoring stack pointer
399.     jr    ra                       # return to caller

```

7.2 Testcases and Output

Test Case 1 — Identity Matrix (6×6)

The screenshot shows a RISC-V assembly simulator interface. The main window displays the assembly code for a program that calculates the determinant of a 6x6 matrix. The code is as follows:

```

1  .data
2  # storing matrix dimension N = 6
3  N: .word 6
4
5  # 6x6 input matrix A stored in row-major order
6
7
8  A: .float
9  1,0,0,0,0,0,
10 0,1,0,0,0,0,
11 0,0,1,0,0,0,
12 0,0,0,1,0,0,
13 0,0,0,0,1,0,
14 0,0,0,0,0,1
15
16 X: .float 1.0, 2.0, 3.0, 4.0, 5.0, 6.0
17
18 # output vector Y (size 6 floats = 24 bytes)
19
20 Y: .space 24
21
22 # storage for determinant value
23 det_result: .float 0.0
24
25 # boolean flag: 1 if sum(Y) < det(A), else 0

```

The right panel shows the status of registers and control elements:

Name	Number	Value
ustatus	0	0
fflags	1	0
frm	2	0
fcsr	3	0
uie	4	0
utvec	5	0
uscratch	64	0
uepc	65	0
ucause	66	0
utval	67	0
uip	68	0
cycle	3072	2121
time	3073	212258593
instret	3074	2121
cycleh	3200	0
timeh	3201	410
instreth	3202	0

The bottom panel shows the output of the program:

```

0
-- program is finished running (0) --
1.0 2.0 3.0 4.0 5.0 6.0
1.0
0
-- program is finished running (0) --

```


Test Case 2 — Singular Matrix

EditExecute

riscv1.asm riscv2.asm final.asm riscv3.asm working

```

1 .data
2 # storing matrix dimension N = 6
3 N: .word 6
4
5 # 6x6 input matrix A stored in row-major order
6
7
8 A: .float
9 1.0, 2.0, 3.0, 0.5, 1.5, 2.5,
10 0.5, 1.0, 1.5, 0.2, 0.8, 1.1,
11 1.5, 3.0, 4.5, 0.7, 2.3, 3.6,
12 2.0, -1.0, 0.5, 1.2, 0.4, 0.9,
13 -0.5, 0.3, 1.1, 0.6, 1.7, -0.2,
14 0.8, 1.4, -0.3, 0.9, 0.5, 1.2
15
16 X: .float 1.0, 2.0, 3.0, 4.0, 5.0, 6.0
17
18 # output vector Y (size 6 floats = 24 bytes)
19
20 Y: .space 24
21
22 # storage for determinant value
23 det_result: .float 0.0
24
25 # boolean flag: 1 if sum(Y) < det(A), else 0

```

Line: 7 Column: 2 ☒ Show Line Numbers

MessagesRun I/O

```

38.5 18.400002 56.899998 13.7 13.1 16.0
-5.0966065E-7
0
-- program is finished running (0) --

```

Clear

RegistersFloating PointControl and Status

Name	Number	Value
ustatus	0	0
fflags	1	1
frm	2	0
fcsr	3	1
uie	4	0
utvec	5	0
uscratch	64	0
uepc	65	0
ucause	66	0
utval	67	0
uip	68	0
cycle	3072	2525
time	3073	2122053596
instret	3074	2525
cycleh	3200	0
timeh	3201	410
instreth	3202	0

Test Case 3 — Float-Heavy Matrix

EditExecute

riscv1.asm riscv2.asm final.asm riscv3.asm working

```

1 .data
2 # storing matrix dimension N = 6
3 N: .word 6
4
5 # 6x6 input matrix A stored in row-major order
6
7
8 A: .float
9 1.25, -2.40, 3.75, 0.50, -1.10, 4.20,
10 -0.80, 1.10, 0.95, -3.30, 2.75, 1.60,
11 0.60, -1.70, 2.20, 1.30, -0.90, 0.40,
12 4.70, -2.20, 0.00, 1.20, 3.30, 0.45,
13 -1.50, 2.80, 1.75, -0.60, 2.25, -3.10,
14 0.95, -1.40, 4.60, 3.20, -0.80, 1.75
15
16 X: .float 1.0, 2.0, 3.0, 4.0, 5.0, 6.0
17
18 # output vector Y (size 6 floats = 24 bytes)
19
20 Y: .space 24
21
22 # storage for determinant value
23 det_result: .float 0.0
24
25 # boolean flag: 1 if sum(Y) < det(A), else 0

```

Line: 11 Column: 43 ☒ Show Line Numbers

MessagesRun I/O

```

0
-- program is finished running (0) --
29.399998 14.400001 6.9 24.3 -0.399997 31.25
-516.6622
0
-- program is finished running (0) --

```

Clear

RegistersFloating PointControl and Status

Name	Number	Value
ustatus	0	0
fflags	1	1
frm	2	0
fcsr	3	1
uie	4	0
utvec	5	0
uscratch	64	0
uepc	65	0
ucause	66	0
utval	67	0
uip	68	0
cycle	3072	2527
time	3073	2122478514
instret	3074	2527
cycleh	3200	0
timeh	3201	410
instreth	3202	0

Test Case 4 — Non-Singular Matrix

The screenshot shows the RISC-V IDE interface for Test Case 4. The assembly code is as follows:

```

1 .data
2 # storing matrix dimension N = 6
3 N: .word 6
4
5 # 6x6 input matrix A stored in row-major order
6
7
8 A: .float
9 1.2, 0.5, -0.3, 2.0, 1.1, 0.4,
10 0.8, 1.5, 0.6, -0.2, 0.7, 1.0,
11 -0.1, 0.3, 1.8, 0.5, 0.2, -0.4,
12 2.2, -0.5, 0.7, 1.3, 0.9, 0.6,
13 1.0, 0.2, 0.4, 0.8, 1.7, 0.3,
14 0.6, 1.1, -0.2, 0.9, 0.5, 1.4
15
16 X: .float 1.0, 2.0, 3.0, 4.0, 5.0, 6.0
17
18 # output vector Y (size 6 floats = 24 bytes)
19
20 Y: .space 24
21
22 # storage for determinant value
23 det_result: .float 0.0
24
25 # boolean flag: 1 if sum(Y) < det(A), else 0

```

The output window shows the following results:

```

17.2 14.299999 6.4999995 16.6 16.1 16.7
-11.157581
0
-- program is finished running (0) --

```

The registers window shows the following values:

Name	Number	Value
ustatus	0	0
fflags	1	1
frm	2	0
fcsr	3	1
uie	4	0
utvec	5	0
uscratch	64	0
uepc	65	0
ucause	66	0
utval	67	0
uip	68	0
cycle	3072	2221
time	3073	2122298093
instret	3074	2221
cycleh	3200	0
timeh	3201	410
instreth	3202	0

Test Case 5 — Submission Matrix (given in assignment)

The screenshot shows the RISC-V IDE interface for Test Case 5. The assembly code is as follows:

```

1 .data
2 # storing matrix dimension N = 6
3 N: .word 6
4
5 # 6x6 input matrix A stored in row-major order
6
7
8 A: .float
9 1.0, 2.0, 3.0, 4.0, 5.0, 6.0,
10 2.0, 1.0, 2.0, 1.0, 2.0, 1.0,
11 3.0, 0.0, 1.0, 2.0, 1.0, 0.0,
12 1.0, 2.0, 3.0, 4.0, 3.0, 2.0,
13 2.0, 1.0, 0.0, 1.0, 2.0, 1.0,
14 1.0, 3.0, 2.0, 4.0, 1.0, 5.0
15
16 X: .float 1.0, 2.0, 3.0, 4.0, 5.0, 6.0
17
18 # output vector Y (size 6 floats = 24 bytes)
19
20 Y: .space 24
21
22 # storage for determinant value
23 det_result: .float 0.0
24
25 # boolean flag: 1 if sum(Y) < det(A), else 0

```

The output window shows the following results:

```

0
-- program is finished running (0) --
91.0 30.0 19.0 57.0 24.0 64.0
-416.0
0
-- program is finished running (0) --

```

The registers window shows the following values:

Name	Number	Value
ustatus	0	0
fflags	1	1
frm	2	0
fcsr	3	1
uie	4	0
utvec	5	0
uscratch	64	0
uepc	65	0
ucause	66	0
utval	67	0
uip	68	0
cycle	3072	2427
time	3073	2122410442
instret	3074	2427
cycleh	3200	0
timeh	3201	410
instreth	3202	0